

Chapter 1

Object-Oriented Analysis and Design

이찬근

소프트웨어학부
중앙대학교



OOA & OOD ↔ Structured-oriented Object-Oriented Analysis and Design

객체 지향

Analysis tool과
design tool이

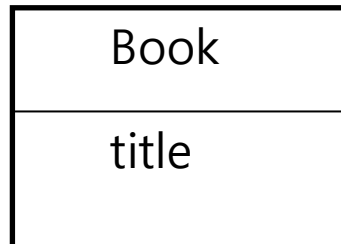
- **Object-Oriented Analysis (OOA)**

- Discover the domain concepts (the objects of the problem domain) 호환 X

(OOD) Object-Oriented Design

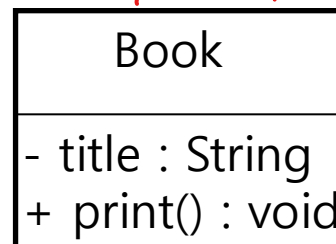
↗ Analysis tool과 design tool이 호환.

- Define software objects and how they collaborate to fulfill the requirements



분석 다이어그램

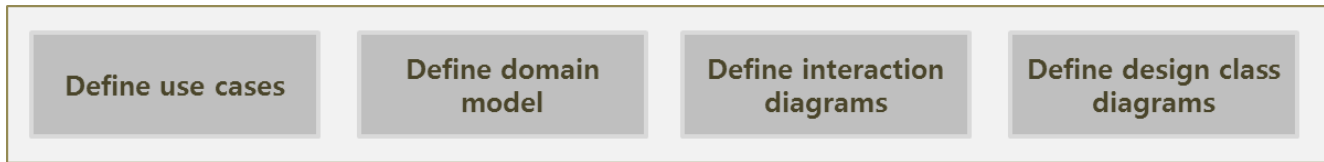
디자인 다이어그램



```
public class Book {
    public void print();
    private String title;
}
```



A Short Example of OOAD : “Dice Game”



- **Define Use Cases**

UC1: Play a Dice Game

- Step1: Player requests to roll the dice
- Step 2: System presents results
- Step 3: If the dice's face value totals seven, player wins; otherwise, player loses.

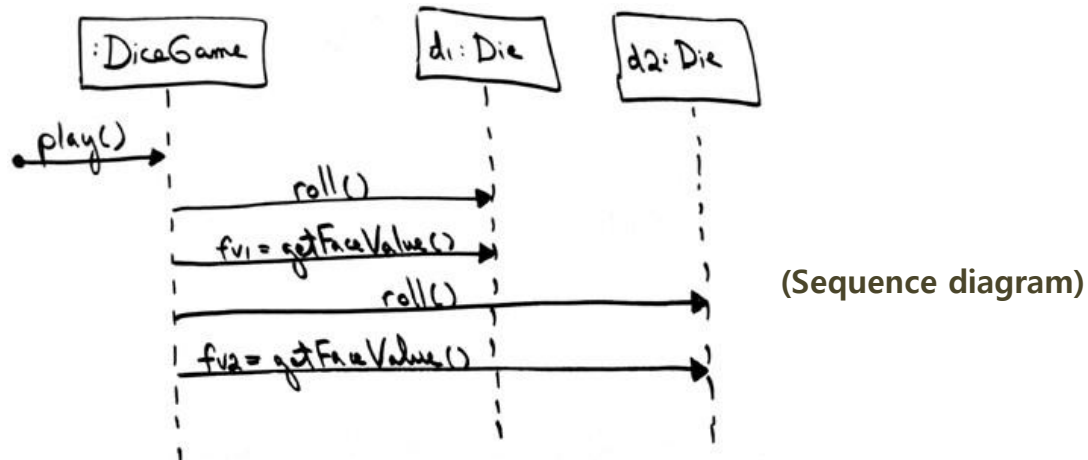
- **Define Domain Model**



Domain model shows the *noteworthy* domain concepts as objects, their attributes, and associations.



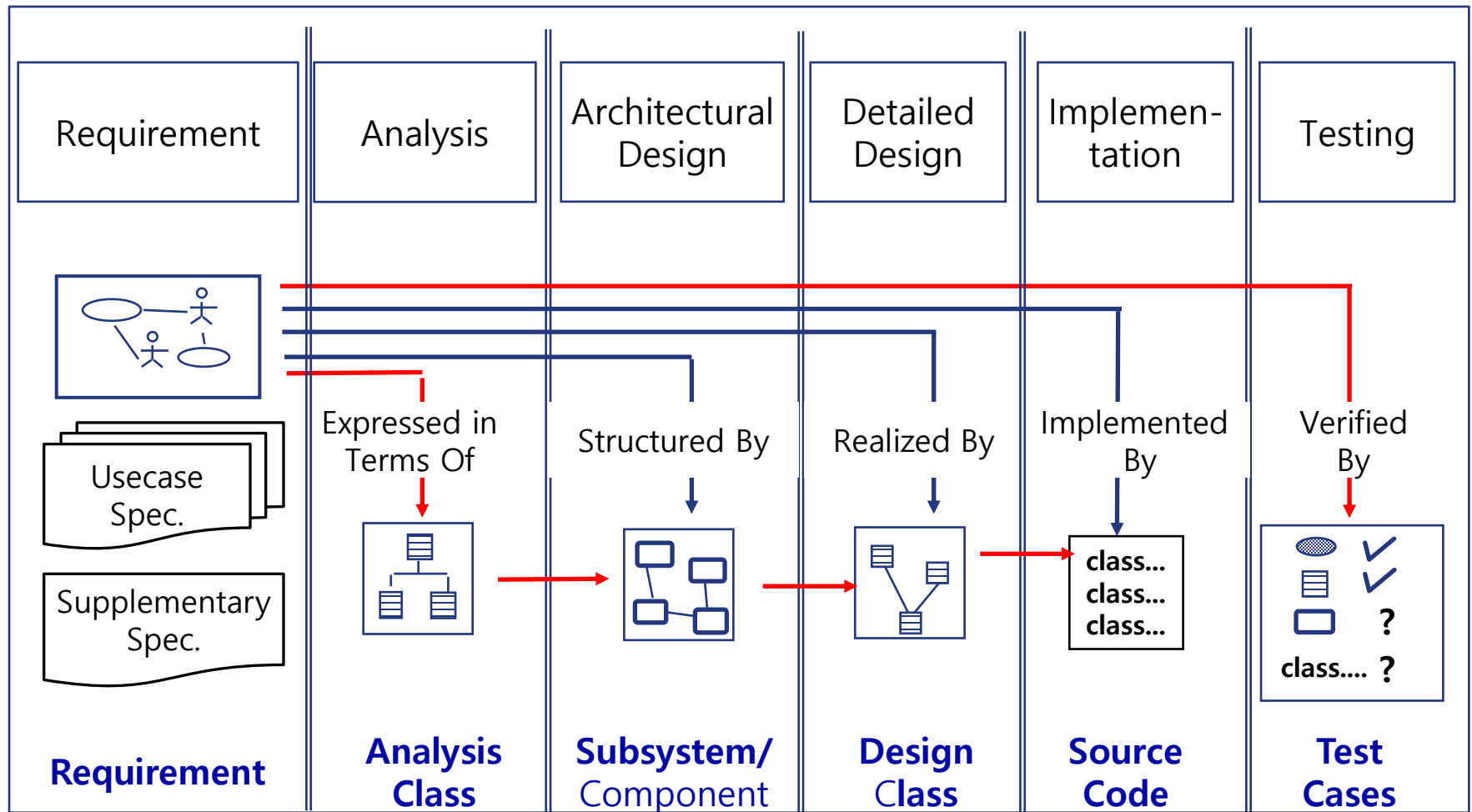
- Define Interaction Diagrams



- Define Design Class Diagrams



Development Process



Three Perspectives to Apply UML

- The same UML notation may be used for three perspectives and types of models.

1. Conceptual perspective *순수하게 도메인 개념만*

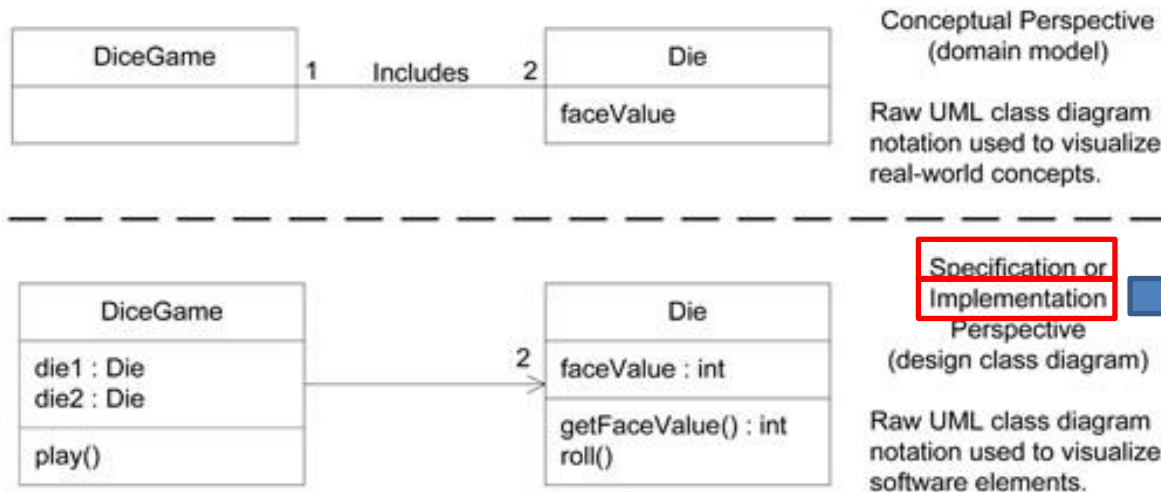
- describe things in a situation of the real world or domain of interest.

2. Specification (software) perspective

- describe software abstractions or components with specifications and interfaces, but no commitment to a particular implementation (e.g., not specifically a class in Java). *특정 구현에만 종속 X / 무엇을 원하는가 / 인터페이스와 행위 명세만 정의*

3. Implementation (software) perspective

- describe software implementations in a particular technology (such as Java). *어떻게 구현하는가 / 구체적인 알고리즘 & 자료구조 / 실제 실행 가능한 코드*



In practice, most software-oriented UML diagramming assumes an **implementation perspective** and the specification perspective is seldom used for design.

